

Section 1: About You

Goals and Objectives

Why do you want to do a GSoC project with Oppia?	I want to contribute to Oppia because of its strong mission to make high-quality education accessible to everyone. The impact-driven nature of the project aligns with my personal motivation to work on software that benefits a large and diverse user base. I have also had the opportunity to contribute as a collaborator in the Developer Workflow team, which has given me valuable exposure to Oppia's development practices and testing infrastructure.	
What do you hope to learn/achieve during GSoC?	During GSoC, I aim to strengthen both my technical and collaborative skills by working on a production-scale open-source project. Technically, I want to: • Gain hands-on experience with end-to-end testing frameworks like Playwright • Understand and resolve test flakiness in CI environments • Improve my skills in debugging complex issues and root cause analysis • Learn how to design and maintain robust test infrastructure • Work with GitHub Actions and CI/CD pipelines From a collaboration perspective, I aim to: • Communicate effectively through well-structured debugging documents • Participate in code reviews and discussions with mentors • Follow best practices for incremental development and clean code contributions	
Which Oppia teams have you collaborated with, and what have you done on those teams?	Developer-Workflow team	Active contributor and collaborator in the Developer Workflow team. Investigated multiple flaky acceptance test issues (e.g., #24578, #24538, #24203, #24033) and identified cases that were no longer reproducible or already resolved by recent PRs. Communicated findings on GitHub and helped

		close stale issues. Successfully migrated 3 tests from E2E to Acceptance tests, contributing to the ongoing transition aimed at improving test stability and maintainability. Contributed to improving test infrastructure by enhancing debugging for acceptance tests. Investigated acceptance test flakiness and explored potential fixes to improve reliability. Currently working on workflow improvements to enhance the PR review process by ensuring contributors are aware they can request reviewers via Oppiabot. Also working on multiple Dev-Workflow issues, with multiple pull requests under review related to acceptance test infrastructure and stability improvements.
	LACE Team	Submitted and merged 4 pull requests addressing UI issues in learner-facing features such as Classroom pages, Learner Dashboard, and Library Fixed layout and responsiveness issues including horizontal overflow, improper text wrapping, and UI misalignment across different screen sizes Improved user experience by ensuring consistent rendering of long text content and proper containment of UI elements Worked on real user-facing bugs affecting Classroom, dashboard views, and lesson cards, contributing to a more polished learning interface Collaborated with reviewers to refine implementations and ensure adherence to Oppia's frontend standards
Contact information	Email : mohak14062005@gmail.com Github : https://github.com/Mohak51234	
Preferred method of communication	I am most comfortable communicating via GitHub (issues, PR discussions) and Google Chat , where I am already part of the Developer Workflow team's communication channels. I also regularly check email and respond quickly.	
Social media handles (optional) <i>This field is optional. We ask for it so that we can tag you in GSoC-related posts if your application is accepted!</i>		

Which timezone will you primarily be in during the summer?	IST (UTC +5:30)
(If you are a student) When are your school holidays?	My college summer break is approximately from May 20 to July 30 , during which I will be available full-time for the project.
What other obligations might you need to work around during the summer?	I do not have any major commitments such as internships, exams, or travel planned during the GSoC period. After July 30, my college will resume; however, the academic workload is moderate, which would allow me to continue contributing consistently to the project.
Planned time commitment	During my summer break (May 20 – July 30), I will be able to dedicate 30–40 hours per week to the project. After my college reopens, I plan to contribute 15–20 hours per week, ensuring steady progress while balancing academic responsibilities. I will maintain a consistent schedule and align my work with the project milestones to ensure timely completion of deliverables.

Project Timeframe

Note: *Oppia will only be offering a single GSoC coding period timeframe this year, starting on **May 25**. All work for Milestone 1 must be completed and submitted by **Jun 26** for internal feedback (with any revisions due by **Jul 10**), and all work for Milestone 2 must be completed and submitted by **Aug 19** for internal feedback (with any revisions due by **Sep 2**). We will not be able to extend these deadlines.*

Coding period	I will adhere to the above deadlines.
----------------------	---

Communication Channels

Note: *The Oppia team places a high emphasis on communication, and we have found that daily contact between contributors and mentors is important for helping keep projects on track. This is why we ask that contributors send short daily updates to the team group chat explaining what they have done, where they are stuck, and what they plan to do next.*

I can commit to sending daily updates to the team group chat, each day I work during the GSoC period.	Yes
--	---

In addition to the above: how often, and through which channel(s), do you plan on communicating with your mentor?	I will communicate daily with my mentor through Google Chat, where I am already part of the Developer Workflow team's group. I will also use GitHub (issues and pull request discussions) for technical updates and async communication. I will share regular progress updates, blockers, and next steps to ensure smooth collaboration.
---	---

Contributor Briefing

Note: A couple of days after the selection results are announced, there will be a mandatory contributor briefing for accepted contributors. Please confirm whether you will be able to attend the briefing.

If accepted, I will be able to attend the contributor briefing at 3 pm UTC on May 1.	• Yes
--	---

Section 2: About Your Project

Project Details

Project title (should match the one on the Project Ideas list)	3.1. Improve Acceptance Test Infrastructure
Project size (should match the options on the Project Ideas list)	Medium (~175 hours)
Why did you choose this project?	I chose this project because it directly aligns with the work I have been doing in Oppia's Developer Workflow team. While working on acceptance test migration and investigating flaky tests, I got exposure to the current challenges in the testing infrastructure, especially around stability and CI reliability. I found this area particularly interesting because it involves not just writing tests, but understanding how tests behave in real environments and why they fail intermittently. The migration to Playwright and efforts to reduce flakiness felt like meaningful problems where I could contribute effectively.

	Since I have already started working on related issues and am familiar with the existing setup, I believe I can ramp up quickly and make consistent progress on this project.
--	---

Required Skills (Project-Specific)

GENERAL	
I can figure out the root cause of an issue, and communicate it effectively using a debugging doc . ¹	W [2026] Debugging Doc for #25317.docx W [2026] Debugging Doc For Issue#24221 Snapshot mismatch.docx
I can debug and fix CI failures / flakes. ²	https://github.com/oppia/oppia/pull/25419 (Merged) https://github.com/oppia/oppia/pull/25733 (Pending Review) https://github.com/oppia/oppia/pull/25785 (Pending Review)
I can make changes to GitHub Actions workflows. ³	Currently working on issue involving this - https://github.com/oppia/oppia/issues/18312 https://github.com/oppia/oppia/pull/25709 My project work will involve specific changes to: <code>.github/workflows/full_stack_tests.yml</code> – to support dual-framework artifact uploads and Playwright browser installation via isolated Node 20 npx path <code>.github/actions/run-a-specific-acceptance-test/action.yml</code> – to integrate Playwright execution into the existing composite action <code>.github/workflows/stress_test_acceptance_test.yml</code> – to extend the matrix strategy for Playwright suites
WEB	
I can write Python code with unit tests. ⁴	PR link - https://github.com/oppia/oppia/pull/25605 (Pending review) Fixes an authorization bug in <code>TopicURLFragmentHandler</code> by updating the ACL decorator. Added unit tests in <code>topic_editor_test.py</code> covering both unauthorized access (normal user receiving 401) and authorized access (topic manager successfully accessing the handler), demonstrating Python backend changes with meaningful test coverage.

¹ To develop this skill: Tackle an issue where the root cause is unclear, identify the root cause in a comment on the issue thread (perhaps with a debugging doc explaining how you figured it out) and submit a PR that fixes it. Fixing CI failures also counts. You might find [this tutorial](#) helpful.

² To develop this skill: See issues labelled “CI breakage” on GitHub, or look at [CI failures in develop](#) (or  signs [here](#)) and figure out how to fix them. [This tutorial](#) (for Web) might also be useful.

³ To develop this skill: Please see the list of issues for the [dev workflow team](#). However, note that this skill is not needed for most projects and there are limited issues available for it. If this affects your project, [please contact the org admins](#).

⁴ To develop this skill: Some [LaCE](#) and [Contributor Dashboard](#) issues have a backend component, and many [Dev Workflow](#) issues do too. Focus on issues that aren’t marked as “backlog”. This is also a good issue to gain experience with backend tests: [#21308](#).

I can write or modify acceptance tests.⁵	https://github.com/oppia/oppia/pull/24832 (Merged) https://github.com/oppia/oppia/pull/24909 (Merged) https://github.com/oppia/oppia/pull/25286 (Merged) In these PR's, I worked on converting existing e2e tests into the acceptance test framework, which involved understanding the structure of CUJ-based tests, updating selectors, and handling asynchronous behavior to ensure stable execution. This gave me familiarity with the acceptance test setup and challenges around test reliability and synchronization, which are directly relevant to the Puppeteer to Playwright migration. Enhanced acceptance test debugging by improving the “wait for element to be clickable” functionality to provide actionable error messages (e.g., identifying if an element is disabled, blocked, or not in the viewport). https://github.com/oppia/oppia/pull/25371 (Merged)
---	---

(Optional) Oppia Community References

Who you worked with	Context in which you worked with them

Section 3: Proposal Details

Note: This section is largely adapted from [Oppia TDD Template \(Dec 2024\) -- PLEASE MAKE A COPY](#). We ***strongly recommend reading that doc***, since it has detailed explanations of how to fill out the various sections. It also contains links to sample TDDs, so that you can see what others have done in the past.

⁵ To develop this skill: Solve part of issue [#21646](#) by covering the CUJs for one or more sets of users.

Problem Statement

Target Audience	Developers and contributors working on Oppia's acceptance test infrastructure and CI workflows, especially those writing, maintaining, or debugging CUJ-based end-to-end tests. The QA team also relies on acceptance test results for release validation and needs to be able to trust them as a stable signal.
Core User Need	Oppia requires a stable, low-flake acceptance testing system that developers can debug efficiently and that QA teams can trust for release validation. Test outcomes should reliably reflect real regressions rather than infrastructure-related noise, enabling confident and accurate release decisions.

Section 3.1: WHAT

Key User Stories and Tasks that will be implemented

#	Title	User Story Description (role, goal, motivation) <i>"As a ..., I need ..., so that"</i>	List of tasks needed to achieve the goal (this is the "User Journey")	Links to mocks / prototypes, and/or PRD sections that spec out additional requirements.
1	Run acceptance tests reliably during and after migration to Playwright	As a contributor maintaining acceptance tests, I need acceptance tests to remain reliable and continue passing throughout the migration so that I can trust test results at every stage and catch real regressions without worrying about framework-related instability.	Contributor assigns or updates the framework configuration for a test suite	https://github.com/oppia/oppia/issues/24715
			Contributor runs acceptance tests locally and verifies that each test suite executes using the correct framework	
			Contributor updates or adds tests, ensuring they are associated with the intended framework	
			Contributor submits changes, and CI runs all acceptance tests using the configured framework for each suite	
			Contributor runs acceptance tests after migration is complete and verifies all suites execute using Playwright with no Puppeteer dependency remaining	
2	Write and	As a contributor adding	Contributor creates a new	https://github.com/oppia/oppia

	maintain acceptance tests using Playwright	new acceptance tests, I need to be able to write new tests in Playwright while migration is ongoing, so that I am always working in the target framework. After migration is complete, I can add and maintain tests only in Playwright.	acceptance test using Playwright in the designated test directory Contributor runs acceptance tests locally and verifies that both new (Playwright) and existing test suites execute successfully Contributor updates or adds new tests in Playwright while existing legacy tests continue to run Contributor submits changes, and CI runs all acceptance tests using the appropriate framework for each suite After migration is complete, contributor adds and maintains acceptance tests only using Playwright	/issues/24715
3	Debug failing acceptance tests using always-available CI artifacts	As a contributor debugging acceptance tests, I need all debugging artifacts to always be present after every CI run so that I can diagnose failures without repeated trial-and-error.	Contributor pushes code changes and triggers CI acceptance tests Contributor observes test results and identifies failing tests Contributor opens a failing test and inspects the failure details All relevant debugging artifacts (logs, screenshots, recordings, snapshot diffs) are consistently available for every failure. Contributor uses these artifacts to understand and diagnose the root cause of the failure	https://github.com/oppia/oppia/issues/23776 https://github.com/oppia/oppia/issues/23155
4	Detect and prevent method	As a contributor writing acceptance tests, I need that even when my test helper	Contributor writes a test helper function with a name that conflicts with an existing one	https://github.com/oppia/oppia/issues/22539

	override conflicts in acceptance test helpers	functions have similar names to existing ones, they should not silently override each other. If an override occurs, I should see a clear error so I can identify the conflict and rename the method in my test.	The system surfaces a clear and actionable error identifying the conflict Contributor renames the conflicting method in their test Contributor re-runs the test and verifies it executes correctly	
5	Provide detailed debugging information for non-clickable elements in acceptance tests	As a contributor writing or debugging acceptance tests, I need clear and specific reasons why an element is not clickable, so that I can quickly diagnose UI interaction failures without guessing whether the issue is due to visibility, overlap, or disabled state.	Contributor runs an acceptance test involving UI interaction Test waits for an element to become clickable If the element is not clickable within the timeout: The system detects the failure and analyzes the element state (e.g., hidden, disabled, or covered) System fails with a detailed error message including: • selector of the element • reason why it is not clickable • relevant DOM/debug context Contributor uses this information to : fix selector / timing / UI state issue Contributor re-runs test and verifies successful interaction	https://github.com/oppia/oppia/issues/20142
6	Detect and fail tests on server-side errors during acceptance test execution	As a contributor running full-stack acceptance tests, I need server-side errors to be detected and surfaced in test output, so that I can identify backend failures immediately instead of	Contributor runs acceptance tests locally or in CI During test execution, backend requests are made as part of the test flow If any server error occurs (e.g., 5xx, exceptions): The system detects the	https://github.com/oppia/oppia/issues/21644

		misinterpreting them	failure and captures relevant error details: • API endpoint • request/response • stack trace (if available)	
			System logs server errors clearly in test output	
			Contributor reviews logs and identifies backend issue	
			Contributor fixes issue or updates test accordingly	
			Contributor re-runs tests and verifies no server errors occur	
7	Detect and provide actionable errors for test runtime failures and hanging processes	As a contributor running acceptance tests in CI, I need clear diagnostics when tests hang, fail to exit, or browser sessions disconnect, so that I can identify root causes like async leaks, crashes, or infrastructure issues without wasting debugging time.	Contributor pushes changes or runs acceptance tests locally/through CI During test execution, the system monitors test lifecycle, including: • test completion • browser session status • background async operations If a runtime issue occurs (e.g., test hangs, does not exit, or browser disconnects): The system detects the abnormal behavior and captures relevant diagnostics The system surfaces a clear and actionable error message, including: • indication of the failure type (e.g., hanging process, browser disconnect) • relevant logs or stack traces • possible causes (e.g., unclosed async operations,	https://github.com/oppia/oppia/issues/22251 https://github.com/oppia/oppia/issues/22193

			connection issues)	
			Contributor reviews the error output, identifies the likely root cause and updates the test or underlying code (e.g., fixes async leaks, stabilizes browser interactions)	
			Contributor re-runs the tests and verifies that: - tests complete successfully - no hanging processes or unexpected disconnections occur	
8	Run acceptance tests with confidence that failures reflect real regressions.	As a contributor running acceptance tests, I need test results to be consistent and reliable so that when a test fails, I can trust it reflects a real issue rather than intermittent infrastructure noise.	Contributor identifies a flaky acceptance test	https://github.com/oppia/oppia/issues/23432 https://github.com/oppia/oppia/issues/23106
			Contributor runs the stress test workflow on the targeted test	
			Contributor observes repeated executions and identifies inconsistent behavior	
			Contributor inspects logs, snapshots, or recordings from these runs to understand the issue	
			Contributor applies fixes and re-runs the stress test workflow to verify stability	
9	Ensure complete acceptance test coverage for existing user flows	As a contributor working on acceptance tests, I need all existing user flows to be covered by acceptance tests so that I can confidently modify and extend functionality without relying on legacy e2e tests.	Contributor identifies a feature or user flow to work on	https://github.com/oppia/oppia/issues/23871
			Contributor locates an existing acceptance test covering that flow	
			Contributor reviews the test to understand expected behavior	
			Contributor updates or	

			extends the test as needed	
			Contributor runs acceptance tests locally and verifies behavior	
			CI runs all acceptance tests and confirms no regressions	
			Contributor debugs failures using available testing tools and artifacts	
			Contributor continues development without needing legacy e2e tests	

Testing Plan

Critical user journeys

User Type	User Story Description	Test definition		
		Setup (ends with a user of the correct type getting logged in)	Test steps	Test expectations
1.Contributor (acceptance tests)	Run acceptance tests correctly during dual-framework migration	Local development environment set up, dependencies installed, and test user authenticated in the Oppia dev server.	Mark sample test suites in <code>acceptance.json</code> with their respective framework	Test suites execute using the configured framework without fallback; incorrect configuration results in a clear error message.
			Run acceptance tests using <code>python -m scripts.run_acceptance_tests</code>	
			Verify that the correct framework (Puppeteer/Playwright) is used for each suite based on configuration	
			Check logs to confirm correct execution path	
2.Contributor (CI/testing)	Debug failed acceptance tests using CI artifacts	PR branch pushed, CI workflow triggered, and contributor has access to CI results.	Run acceptance tests through CI workflow	Failure produces complete and actionable artifacts including logs, snapshots (diff/new), and screen recordings for debugging.
			Trigger a failure scenario (e.g., known flaky test or controlled failure)	
			Open the failing test and inspect the failure details	

			Verify that all debugging artifacts (logs, screenshots, recordings, snapshot diffs) are automatically present without any extra steps	
3.Contributor (acceptance tests)	Validate non-fast-fail snapshot behavior	Local environment set up with baseline snapshots available for the target test suite.	Introduce a controlled snapshot mismatch in a test step	Test continues execution after mismatch and fails at the end with a snapshot summary; updated snapshots are available as artifacts.
			Execute the full acceptance test flow	
			Observe whether remaining test steps continue execution	
			Inspect final failure output and uploaded artifacts	
4.Contributor (debugging flakes)	Verify a previously flaky test now passes consistently after fixes	A test that was previously identified as flaky and has been fixed; stress test workflow available.; stress test workflow available.	Run the stress test workflow on the targeted test with multiple runs	Test passes consistently across all stress test runs with near-zero failure rate; the fix does not introduce any regressions in other tests.
			Observe that the test passes consistently across all runs	
			Verify no intermittent failures occur under the same conditions that previously caused flakiness	
5.Contributor / Reviewer	Confirm migration completion and cleanup	Migration branch completed with all acceptance tests passing in CI.	Verify all acceptance tests run through Playwright configuration	All acceptance tests run through Playwright; legacy Puppeteer dependencies are removed or clearly deprecated; CI remains stable.
			Confirm acceptance test runner executes correctly using <code>python -m scripts.run_acceptance_tests</code>	
			Check that CI workflows complete successfully	
			Ensure Puppeteer-specific dependencies and scripts are removed or isolated	

Technical Requirements

Additions/Changes to the Web Server Interface

#	Endpoint URL	Request type (GET, POST, etc.)	New / Existing	Description of the request/response contract (and, if applicable, how it's different from the previous one). For new endpoints, include which access control decorator (e.g. "can_play_exploration", "open_access", etc.) will be used.
---	--------------	--------------------------------	----------------	---

1.				
2.				

New/Changed Calls by the Web Frontend /Android Client to the Web Server Interface

#	Endpoint URL	Request type (GET, POST, etc.)	Description of why the new call is needed, or why the changes to an existing call are needed
1.			
2.			

Web Server Storage Model Additions/Changes

#	Datastore model	Description of changes	How will existing data be handled?
1.			
2.			

UI Screens/Components

#	ID	Description of new UI component	i18n required?	Mock/spec links	What A11y support will be implemented?
1.					
2.					

Data Handling and Privacy

#	Type of data	Description	Why do we need to store this data?	Anonymized?	Can the user opt out?	Wipeout policy	Takeout policy
1.							
2.							

Connection to Existing Work

This project builds on Oppia's existing acceptance testing setup, which currently uses Puppeteer and is managed through Python scripts and CI workflows.

Relevant components in the codebase

1. **Acceptance suite registry and configuration**
<https://github.com/oppia/oppia/blob/develop/core/tests/ci-test-suite-configs/acceptance.json>
This file defines acceptance test suites and maps them to their corresponding modules. It serves as the central configuration for selecting and organizing test execution.
2. **Acceptance test runner**
https://github.com/oppia/oppia/blob/develop/scripts/run_acceptance_tests.py
This script is responsible for orchestrating acceptance test execution, including selecting test suites and invoking the appropriate test processes.
3. **Acceptance test process orchestration**
<https://github.com/oppia/oppia/blob/develop/scripts/servers.py>
This layer manages test execution environments and coordinates how acceptance tests are launched and run.
4. **Puppeteer test configuration**
<https://github.com/oppia/oppia/blob/develop/core/tests/puppeteer-acceptance-tests/jest.config.js>
This configuration defines how Puppeteer-based acceptance tests are executed within the testing framework.
5. **Acceptance test specifications (CUJ definitions)**
<https://github.com/oppia/oppia/tree/develop/core/tests/puppeteer-acceptance-tests/specs>
These files contain the actual acceptance test cases, representing critical user journeys and expected system behavior.
6. **Shared utilities for acceptance tests**
<https://github.com/oppia/oppia/tree/develop/core/tests/puppeteer-acceptance-tests/utilities>
These utilities provide reusable helpers for interacting with the UI, managing test setup, and performing assertions across test suites.
7. **CI workflows for acceptance testing**
https://github.com/oppia/oppia/blob/develop/.github/workflows/full_stack_tests.yml
This workflow runs acceptance tests in CI and uploads debugging artifacts such as logs, screenshots, and recordings.
8. **Stress test workflow for flake detection**
https://github.com/oppia/oppia/blob/develop/.github/workflows/stress_test_acceptance_test.yml
This workflow repeatedly runs acceptance tests to identify flaky behavior and validate test stability.
9. **Composite CI action for acceptance tests**
<https://github.com/oppia/oppia/blob/develop/.github/actions/run-a-specific-acceptance-test/action.yml>
This action handles execution details such as logging, artifact generation, and test invocation in CI.
10. **Ongoing work tracked in issues**
<https://github.com/oppia/oppia/issues/24715>
<https://github.com/oppia/oppia/issues/23871>

Other Requirements

Since this project involves modifying test infrastructure and CI workflows, care needs to be taken to avoid breaking existing test execution.

Some important considerations:

- **Maintaining compatibility during migration:**
Puppeteer and Playwright tests will need to run together for some time, so changes should not break existing test suites.
- **Keeping CI stable:**
Acceptance tests are already resource-heavy, and stress tests run multiple times. Any changes should avoid increasing flakiness or significantly slowing down CI.
- **Incremental migration:**
Tests should be migrated gradually so that the system remains usable and stable at every stage of the transition.
- **Shared utilities:**
Some helper functions are reused across test suites, so temporary multi-framework support can introduce conflicts if not handled carefully. These may include:
 - Type and interface mismatches (e.g., Puppeteer's ElementHandle/Page vs Playwright's Locator)
 - Differences in utility behavior (e.g., waiting, retries, element lookup)
 - Mismatches between test configuration and execution logic
 - Inconsistencies in CI artifact generation (logs, screenshots, recordings)
- **Debugging support:**
Improvements like better logs, snapshot handling, and recordings should integrate with existing workflows rather than introducing separate systems.
- **Dependencies:**
Playwright will be introduced as the new testing framework. Any additional dependencies required for running Playwright or CI integration will be kept minimal and aligned with the existing setup.

Overall, the focus will be on making changes that are incremental, well-tested, and do not disrupt other contributors.

Section 3.2: HOW

Existing Status Quo

Oppia already has a structured acceptance test execution pipeline, but it is currently Puppeteer-centric and shows reliability issues, especially in CI. The current design also makes migration and debugging more difficult than necessary.

Current technical baseline:

1. Acceptance suite source-of-truth is `acceptance.json`, and suite entries currently point to Puppeteer-based test modules. A separate WebdriverIO test suite also exists, tracked in `e2e.json` with 15 suites pointing to `core/tests/webdriverio_desktop/` and `core/tests/webdriverio/`, configured via `wdio.conf.js`. These are being incrementally migrated to Puppeteer acceptance tests, but may not be fully removed by GSoC start.
2. Local execution is handled through `scripts/run_acceptance_tests.py`, which compiles Puppeteer acceptance TypeScript and invokes the test runner.
3. Suite dispatch and validation are tied to configuration in `acceptance.json` and executed using Puppeteer-based Jest setup.
4. CI workflows (e.g., `full_stack_tests.yml`, stress tests) rely on this same configuration and execution flow.
5. Shared utilities still use Puppeteer-specific constructs such as `ElementHandle`, which tightly couples tests to the framework.

This setup works, but different parts of the system are closely tied to Puppeteer. Because of this, migrating tests gradually is difficult, and changes in one part can affect others, increasing the risk of breaking CI.

Evidence that the current system is not ideal:

1. Existing issues track flakiness, snapshot mismatches, browser instability, and migration effort.
2. CI runs sometimes show instability (e.g., mobile recording issues, intermittent failures).
3. The requirement to fix persistent flakes indicates that reliability issues remain even after regular maintenance.

Barriers / dependencies to change:

1. `acceptance.json` is deeply integrated with both local execution and CI sharding, so any schema changes must remain backward-compatible.
2. Shared helper functions used across tests can introduce conflicts if modified during dual-framework support.
3. CI artifact handling and workflows depend on the current execution structure, so migration must preserve logging, snapshots, and debugging outputs.

Solution Overview

This project improves Oppia's acceptance testing infrastructure by migrating from Puppeteer to Playwright while maintaining CI stability and improving debugging reliability. The migration follows a phased approach that preserves existing behavior during transition and results in a Playwright-only system.

1. Migration Strategy (Abstract View)

The migration will proceed in the following stages:

- **Initialize isolated Playwright runtime**
 - Install Node 20 binary under `oppia_tools` as a parallel installation alongside the existing Node 16 setup
 - Add Playwright-specific constants to `scripts/common.py` (`PLAYWRIGHT_NODE_PATH`, `PLAYWRIGHT_NPX_BIN_PATH`, etc.)
 - Add `@playwright/test` as a dependency in an isolated Playwright project under `core/tests/playwright-acceptance-tests/`
 - Install Playwright npm dependencies and Chromium browser via `install_playwright_dependencies()`, called automatically on first Playwright suite run
 - Add initial smoke spec (`home-page-loads.spec.ts`) to validate the setup
- **Enable framework-aware execution**
 - Extend configuration to support both Puppeteer and Playwright (refer to Point 3: Configuration and Execution Model)
 - Update `servers.py` and `run_acceptance_tests.py` to route execution to either Jest (Puppeteer) or Playwright based on the framework field
- **Incremental migration of test suites**
 - Migrate CUJs from `puppeteer-acceptance-tests/specs/` to `playwright-acceptance-tests/specs/` with minimal behavioral changes
 - Migrate any remaining WebdriverIO-based e2e tests to Playwright acceptance tests
 - Validate each migrated suite using CI and stress workflows before proceeding
- **Transition to Playwright-only system**

- Remove Puppeteer-specific code, dependencies, and configuration
 - Simplify execution workflows
 - **Stabilization and optimization**
 - Improve debugging, recordings, and snapshot handling
 - Resolve flaky tests and document fixes
-

2. Dependencies and Compatibility

Oppia currently uses Node 16.13.0 (pinned via `NODE_VERSION = '16.13.0'` in `scripts/common.py`). All current Playwright versions require Node ≥ 18 , with Node 20+ being the officially recommended version as Node 18 support is now deprecated in Playwright (deprecated in v1.54).

Proposed approach: Rather than upgrading Oppia's global Node version, an isolated Node 20 runtime will be installed under `oppia_tools` and used exclusively for Playwright. This keeps Oppia's existing Node 16 setup completely unchanged for all current scripts and CI jobs. Four additive constants — `PLAYWRIGHT_NODE_VERSION`, `PLAYWRIGHT_NODE_PATH`, `PLAYWRIGHT_NPM_BIN_PATH`, and `PLAYWRIGHT_NPX_BIN_PATH` — will be added to `scripts/common.py` as reference paths only. These constants are **never added to `os.environ['PATH']` globally** — Node 20's bin directory is only prepended to PATH inside the subprocess environment in `managed_acceptance_tests_server()` when dispatching a Playwright suite, ensuring no other Oppia script ever resolves `node` to Node 20 unexpectedly.

The Playwright project's `node_modules` directory is located at `core/tests/playwright-acceptance-tests/node_modules/`, completely separate from Oppia's root `node_modules/`. This ensures no dependency conflicts between Playwright's packages and Oppia's existing Node 16 dependencies.

The isolated Playwright project introduces one new dependency: `@playwright/test` (test runner and browser automation framework).

3. Configuration and Execution Model

Test execution will remain configuration-driven, with no required CLI changes. The existing `acceptance.json` will be extended to include a `framework` field per suite entry. Suites with `"framework": "playwright"` will be dispatched to the local Playwright test runner binary at `core/tests/playwright-acceptance-tests/node_modules/.bin/playwright`, executed under the isolated Node 20 runtime. Suites with `"framework": "puppeteer"` will continue using Jest as before.. All existing suite entries will be updated upfront to include `"framework": "puppeteer"` explicitly, ensuring every suite has a framework field. New Playwright suites use `"framework": "playwright"`.

The CLI entry point `python -m scripts.run_acceptance_tests` remains stable throughout.

Full configuration format and routing code are detailed in Sections 13.2.1–13.2.3.

4. Directory Structure and Isolation

To prevent conflicts during migration:

`core/tests/puppeteer-acceptance-tests/`

`core/tests/playwright-acceptance-tests/`

- Both frameworks operate independently
 - Shared utilities remain unchanged unless necessary
 - Puppeteer directory will be removed after migration completion
-

5. Recorder Migration

`puppeteer-screen-recorder` will be replaced with Playwright's built-in video recording and trace collection, configured globally via `playwright.config.ts`. Recording lifecycle will be managed automatically by the framework per test, eliminating all manual recorder setup in `base-user.ts`. The Puppeteer version of `base-user.ts` remains unchanged during migration.

Full implementation is detailed in Section 13.4.

6. Incremental Migration Approach

Migration will prioritize moving all tests from Puppeteer to Playwright with **minimal changes**, ensuring correctness while reducing conflicts between frameworks. Existing test logic, assertions, and structure will be preserved. CI and stress workflows will be used to validate correctness after each suite is migrated.

There are 137 CUJ spec files across 28 user types.

In this project we would migrate tests of logged-in-learner, logged-out-learner and exploration-creator as described in milestone tables.

Migration order: We will be migrating core user journeys like logged-in-learner, logged-out-learner and exploration-creator specs first.

Migration Prioritization: The primary goal is to achieve full Playwright coverage as quickly as possible. Refactoring tasks such as replacing `ElementHandle` with `Locator` and replacing manual waits with Playwright's auto-waiting mechanisms will be **deferred until after migration is complete**, so initial efforts focus entirely on correctness rather than code quality.

Shared Utilities Conflict Prevention: Codeowners would ensure that the PRs changing the shared methods (in puppeteer and playwright) makes sure that both copies are in sync.

Key Adjustments During Migration:

- Gradual transition from `ElementHandle` to `Locator` (post-migration phase)
- Replace manual waits with Playwright's auto-waiting mechanisms
- Update selectors only when necessary

Post-Migration Cleanup: Once all tests are migrated, shared utilities will be refactored to remove Puppeteer-specific logic, all remaining `ElementHandle` usage will be replaced with `Locator`, and all transitional compatibility code will be removed.

7. CI Workflow Updates

CI workflows will be updated to support both frameworks. Browser installation is handled automatically by `install_playwright_dependencies()` within `run_acceptance_tests.py` — no separate CI install step is needed. The existing composite GitHub Action (`run-a-specific-acceptance-test/action.yml`) will be updated to support both frameworks. Debugging artifacts — logs, screenshots, snapshot diffs, videos, and traces — will be standardized and uploaded on every CI run. Puppeteer artifact uploads will be preserved during the migration period. Retry behavior and overall runtime will be kept stable.

Full YAML changes are detailed in Section 13.3.

8. Stress Test Workflow Enhancements

The existing `stress_test_acceptance_test.yml` matrix strategy will be extended to include Playwright suites, ensuring both frameworks can run within the same workflow during migration. Tests will be executed repeatedly across multiple iterations, with Playwright traces and videos collected per run. Flake rates will be measured and compared across runs to identify unstable tests.

Full workflow changes are detailed in Section 13.5.

9. Debugging and Developer Workflow

Contributors will be able to debug failing acceptance tests both locally and in CI using Playwright's built-in tooling. Locally, tests can be run in headed mode or stepped through interactively using Playwright Inspector, with Trace Viewer used to inspect DOM snapshots, network activity, and console logs at the exact point of failure. In CI, artifact bundles containing videos, traces, and screenshots will be available on every run for download and local analysis. A debugging guide documenting this full workflow will be published to the [oppia-web-developer-docs](#) repository, addressing tracking issues #16136, #22319

Full commands and CI artifact paths are detailed in Section 13.6.

10. Snapshot non-fast-fail fix(issue #23776)

The current behavior in `expectScreenshotToMatch` throws immediately on a mismatch, aborting remaining test steps. This will be fixed in Milestone 2 by updating `fixtures.ts` to collect snapshot errors in an instance-level array instead of throwing immediately, and checking this array in fixture teardown after all test steps complete. This ensures the full test flow always executes before any snapshot failure is reported. New snapshots taken during the run will be uploaded as CI artifacts.

Full implementation is detailed in Section 13.7.

11. Flake Detection and Root Cause Analysis

For each flaky test, the following process will be used. The debugging process starts directly on CI using the stress test workflow (`stress_test_acceptance_test.yml`), since many flakes are caused by environment differences — such as slower CPUs, network latency, or parallel job contention — that do not reproduce locally. Local reproduction will be attempted where possible, but CI stress runs are the primary validation mechanism. Root cause analysis uses Playwright Trace Viewer artifacts from these CI runs. Fixes will be minimal and targeted, validated by re-running the stress workflow until the failure rate reaches near zero.

Full triage methodology is detailed in Section 13.6.

12. Scope and Guardrails

This project focuses on:

- Migration of acceptance tests from Puppeteer to Playwright
- Supporting dual-framework execution during transition
- CI stability and reliability improvements
- Debugging and observability enhancements

Additionally:

- If WebdriverIO-based e2e tests are not fully removed before the start of the coding period, they will be migrated to Playwright as part of this project
- Any remaining e2e tests will be converted into acceptance tests to ensure consistency with Oppia's testing infrastructure

Out of scope:

- Adding new acceptance tests except where required to maintain adequate test coverage or address gaps identified during migration or evaluation of flaky tests
 - Large-scale refactoring beyond migration needs
-

Implementation Details (How the Migration Will Be Done)

This section outlines the concrete implementation steps aligned with Oppia's current acceptance testing infrastructure, ensuring compatibility with existing workflows while enabling incremental migration to Playwright.

13.1 Playwright Setup

An isolated Node 20 runtime will be installed under `oppia_tools` exclusively for Playwright. This does not affect the existing Node 16 installation or any current build, lint, or test scripts.

Node 20 installation:

Node 20 installation is handled automatically as part of Oppia's existing setup flow. A dedicated `install_playwright_node()` function will be added to `scripts/install_third_party_libs.py`, which is called during the standard `python -m scripts.install_third_party_libs` setup step alongside the existing `install_node()` call. Contributors do not need to run any additional commands – Node 20 is installed automatically when setting up the development environment.

Ref - **Appendix** section at the end

Isolated Playwright project setup:

A standalone Playwright project will be created under `core/tests/playwright-acceptance-tests/` with its own `package.json` and dependencies.

Installation is handled automatically – contributors do not need to run any manual commands.

When `python -m scripts.run_acceptance_tests` is called with a Playwright suite, `install_playwright_dependencies()` runs automatically before the test starts. This function:

- Runs `npm ci` using `PLAYWRIGHT_NPM_BIN_PATH` (the isolated Node 20 npm binary) to install `node_modules/` including the `@playwright/test` package and the `node_modules/.bin/playwright` binary
- Runs `playwright install chromium` using the local `node_modules/.bin/playwright` binary – not `npx` – to install the Chromium browser to `~/.cache/ms-playwright/`

The `projects` array is controlled by `process.env.MOBILE` so that the `--mobile` flag passed by CI runs only the mobile viewport, matching Puppeteer's existing behavior

13.2 Test Runner Integration and Framework Routing

Currently, acceptance tests are executed via:

`run_acceptance_tests.py` → `servers.managed_acceptance_tests_server` → Jest (Puppeteer)

To support incremental migration, execution will be extended to support both Puppeteer and Playwright.

The `compile_test_ts_files()` function in `run_acceptance_tests.py` currently compiles Puppeteer TypeScript to `puppeteer-acceptance-tests/build/` before Jest can run it. For Playwright suites this step is unnecessary. During the dual-framework period, `compile_test_ts_files()` will still be called because at least one Puppeteer suite will be running. Once all suites are migrated to Playwright and Puppeteer is removed, `compile_test_ts_files()` and the `shutil.copypath` for the `data/` folder will be removed from `run_acceptance_tests.py`

13.2.1 Extending Acceptance Test Configuration

Each suite entry will include a `framework` field:

Ref - Appendix section at the end

Backward compatibility:

All existing suite entries in `acceptance.json` will be updated upfront to include `"framework": "puppeteer"` explicitly. New Playwright suites would use `"framework": "playwright"`

13.2.2 Framework-aware Suite Mapping in `servers.py`

The suite mapping will be extended to include framework metadata:

```
available_suites[suite['name']] = {  
    'module': suite['module'],  
    'framework': suite['framework']  
}
```

13.2.3 Execution Routing Based on Framework

Routing logic inside `servers.py` will use the isolated Playwright constants rather than the default `node_modules` path:

Ref - **Appendix** section at the end

The local `node_modules/.bin/playwright` binary is used directly instead of `npx` to avoid remote version resolution or unintended downloads during CI runs.

13.2.4 Compatibility and Environment Handling

- Existing Puppeteer environment variables remain unchanged
- Playwright configuration is handled via `playwright.config.ts`

13.2.5 Test Coverage for Routing Logic

Tests will be added to the existing `scripts/servers_test.py` and `scripts/run_acceptance_tests_test.py` files

New test methods in `ManagedProcessTests` (`servers_test.py`):

- `test_managed_acceptance_test_server_playwright_suite`— verifies that a suite with `"framework": "playwright"` results in a command containing `playwright` and not `jest`
- `test_managed_acceptance_test_server_playwright_headless`— verifies `HEADLESS='true'` when `headless=True`

- `test_managed_acceptance_test_server_playwright_mobile`— verifies `MOBILE='true'` when ran with `-mobile` flag.
- `test_managed_acceptance_test_server_with_invalid_framework`— verifies `exception` is raised for any framework string other than `playwright` or `puppeteer`

New class `RunAcceptanceTestsTests` (`run_acceptance_tests_test.py`):

- `test_playwright_deps_called_for_playwright_suite` — checks `install_playwright_dependencies` is called if suite framework is `playwright`.
 - `test_run_tests_raises_error_when_suite_not_found` — checks an exception is raised when the suite is not found in the config.
 - `test_install_playwright_dependencies_success` — checks playwright dependencies are installed when `node_modules` aren't already found.
 - `test_install_playwright_dependencies_skips_if_node_modules_exist` — checks playwright dependencies are not installed (by verifying `check_call` is not called) when `node_modules` aren't already found.
 - `test_install_playwright_dependencies_failure` - checks an exception is raised when the `check_call` fails.
-

13.2.6 – Updating `check_tests_are_captured_in_ci.py`

- The script `scripts/check_tests_are_captured_in_ci.py` validates that every `.spec.ts` file on disk has a corresponding entry in `acceptance.json`.
 - It currently scans only `ACCEPTANCE_TEST_SPECS_DIRECTORY` (pointing to `puppeteer-acceptance-tests/specs`).
 - A new constant `PLAYWRIGHT_TEST_SPECS_DIRECTORY` pointing to `core/tests/playwright-acceptance-tests/specs` will be added, and `get_acceptance_test_suites_from_acceptance_directory()` will scan both directories.
 - The `TestSuiteDict` TypedDict will be extended with an optional `framework` field. Corresponding unit tests in `check_tests_are_captured_in_ci_test.py` will be updated to cover both directories.
 - No changes are needed to the `e2e/WebdriverIO` sections of this script — those will be removed entirely once all `WebdriverIO` suites are migrated to `Playwright` and `e2e.json` and `wdio.conf.js` are deleted.
-

13.3 CI Workflow Changes

The existing entry point remains unchanged: `python -m scripts.run_acceptance_tests`

Browser installation is handled entirely within `install_playwright_dependencies()`, which is called automatically by `run_acceptance_tests.py` before any Playwright suite runs. Chromium is installed to the default Playwright cache directory (`~/.cache/ms-playwright/` on Linux), which is writable on both local machines and CI runners. No separate browser install step is needed in the CI YAML — the Python runner handles installation on demand before tests start.

The acceptance test runner internally dispatches Playwright-based suites using the local `node_modules/.bin/playwright` binary, which is installed using the isolated Node 20 npm binary during the `install_playwright_dependencies()` step.

In CI, Playwright tests run under Node 20 even though the CI job's default Node version is 16. This works because `managed_acceptance_tests_server()` passes an explicit `env` dict to the Playwright subprocess with `PLAYWRIGHT_NODE_PATH/bin` prepended to `PATH` and `NODE` pointing to the Node 20 binary. The Playwright subprocess therefore resolves `node` to Node 20 while the parent CI job and all other processes (dev-server, webpack, yarn) continue using Node 16 unchanged. No global Node version switch occurs at any point in the CI job.

Node 20 is available on CI runners because `install_playwright_node()` is called as part of the standard `merge-develop-and-set-up-dependencies` action that already runs at the start of every acceptance test CI job. This installs Node 20 to `oppia_tools/node-20.11.1/` before any tests begin, requiring no separate CI step.

Additional CI changes:

- Integrate Playwright into the existing composite GitHub Action (`run-a-specific-acceptance-test/action.yml`) by adding Playwright artifact upload steps and `continue-on-error: true` to the existing Puppeteer video upload step, since the Puppeteer video path does not exist for Playwright suites
- Maintain compatibility with existing Python dependencies and Yarn caching
- Keep Puppeteer artifact uploads active during the migration period

Artifacts:

- Upload Playwright outputs: `test-results/`, traces, videos, screenshots
 - Continue Puppeteer artifact uploads during migration
-

13.4 Recorder Migration

Current system:

- Uses PuppeteerScreenRecorder with environment-based toggles

Migration approach:

- Replace with Playwright native features:

- video recording
 - trace collection
 - Configure via Playwright config
 - The PuppeteerScreenRecorder setup in `base-user.ts` (inside the `VIDEO_RECORDING_IS_ENABLED` guard in `openBrowser()`) and the corresponding `screenRecorder.stop()` call in `closeBrowser()` will not be present in the Playwright version of the base user utility. Playwright's `playwright.config.ts` will instead set `video: 'on'` globally, and recording lifecycle is managed automatically by the framework per test. The `SIGTERM/SIGINT` handlers added to stop the screen recorder on signal will also be removed from the Playwright version.
 - The Puppeteer `base-user.ts` keeps its recorder unchanged while only the Playwright version omits it.
-

13.5 Stress Test Workflow Changes

Current workflow:

- Uses matrix-based repeated execution via GitHub Actions (`stress_test_acceptance_test.yml`)
- Runs acceptance tests multiple times to detect flaky behavior

Proposed changes:

The stress test workflow requires no matrix changes for framework selection. The framework is already determined by the suite's `acceptance.json` entry – when a Playwright suite name is passed as `test_suite` input, `run_acceptance_tests.py` reads the `framework` field and automatically routes to Playwright. The existing matrix of `run × viewport-type` remains unchanged.

The only change needed is adding a dedicated Playwright artifact upload step so that traces, videos, and reports are available per run for flake analysis:

```
- name: Upload Playwright artifacts
  if: ${{ !cancelled() }}
  uses: actions/upload-artifact@v4
  with:
    name: playwright-results-${{ matrix.run }}-${{ matrix.viewport-type }}
    path: |
      core/tests/playwright-acceptance-tests/test-results/
```

Once migration is complete, a dedicated Playwright-only stress job will be introduced so Playwright flake detection can run independently of Puppeteer.

13.6 Debugging and Flake Analysis

Local debugging workflow:

Run a specific suite in headed mode(default) for interactive inspection:

```
python -m scripts.run_acceptance_tests --suite <suite_name>
```

Step through test actions interactively using Playwright Inspector:

```
core/tests/playwright-acceptance-tests/node_modules/.bin/playwright test --debug
```

Inspect a trace file captured from a previous run:

```
core/tests/playwright-acceptance-tests/node_modules/.bin/playwright show-trace  
test-results/<suite>/trace.zip
```

The Trace Viewer shows DOM snapshots, network calls, console logs, and timing at every test step, allowing precise identification of the failure point without re-running the test.

CI debugging workflow:

1. Download the artifact bundle from the failed run — artifacts are at `test-results/` path in the GitHub Actions run summary
2. Open the trace locally using `playwright show-trace` or on the web in `trace.playwright.dev` using the artifact generated in CI.
3. Compare the failing run's DOM snapshot and network timeline against a passing run to identify what changed
4. Reproduce consistently using the stress test workflow (`stress_test_acceptance_test.yml`) before attempting a fix

Ref - **Appendix** section at the end

Once fixed, a brief debugging doc will be written for the issue following Oppia's debugging doc format, linking the stress test run results as evidence of stability.

13.7 Snapshot and Recording Improvements

Snapshot non-fast-fail fix (issue #23776):

The fix will be implemented across two files, both newly created as part of the Playwright setup:

1. In `core/tests/playwright-acceptance-tests/utilities/base-user.ts`:

```

export class BaseUser {
  readonly page: Page;
  snapshotErrors: Error[] = [];

  constructor(page: Page) {
    this.page = page;
  }

  async expectScreenshotToMatch(name: string): Promise<void> {
    try {
      await expect(this.page).toHaveScreenshot(name);
    } catch (error) {
      this.snapshotErrors.push(error as Error);
    }
  }
}

```

2. In `core/tests/playwright-acceptance-tests/utilities/fixtures.ts`

```

import { test as base, Page } from '@playwright/test';
import { BaseUser } from './base-user';

export const test = base.extend<{ baseUser: BaseUser }>({
  baseUser: async ({ page }, use) => {
    const user = new BaseUser(page);

    // Run all test steps first
    await use(user);

    // After all test steps complete, report any snapshot mismatches
    if (user.snapshotErrors.length > 0) {
      throw new Error(
        `Snapshot mismatches occurred:\n` +
        user.snapshotErrors.map(e => e.message).join("\n")
      );
    }
  },
});

export { expect } from '@playwright/test';

```

All test files will import `test` from this fixtures file instead of directly from `@playwright/test`:

```

// In test specs:
import { test, expect } from '../utilities/fixtures';

```

Since recording lifecycle is managed automatically per test by Playwright, mobile recordings are captured consistently without any additional setup. This replaces the `VIDEO_RECORDING_IS_ENABLED` environment-based toggle that existed in the Puppeteer `base-user.ts`.

13.8 Shared Utilities Lint Check

Codeowners would ensure that the PRs changing the shared methods(in puppeteer and playwright) makes sure that both copies are in sync.

13.9 UserFactory Method Override Fix (issue #22539)

Part 1 – Lint check for duplicate function names:

A new lint check will be added to `scripts/linters/other_files_linter.py`. Looking at `run_lint_checks.py`, the `other_files_linter` is already registered under the 'other' extension branch via `other_files_linter.get_linters(file_cache)` – the new check slots in here with no new registration needed.

The check will:

- Scan all `.ts` files under `core/tests/playwright-acceptance-tests/utilities/user/`
- Extract all `async` method names using the regex pattern `async\s+(\w+)\s*\((`
- Build a map of `{methodName: [files where it appears]}`
- Flag any method name appearing in more than one file with a clear error

```
PLAYWRIGHT_USER_UTILITIES_DIR = os.path.join(
    'core', 'tests', 'playwright-acceptance-tests',
    'utilities', 'user'
)

def check_duplicate_method_names_in_user_utilities(filepath, content):
    """Checks for duplicate method names across user utility files."""
    # Only applies to user utility TypeScript files
    if not filepath.startswith(PLAYWRIGHT_USER_UTILITIES_DIR):
        return None
    if not filepath.endswith('.ts'):
        return None
    # Extract async method names
    method_names = re.findall(r'async\s+(\w+)\s*\(', content)
    return method_names
```

The orchestrating function collects all method names across all user files, finds duplicates, and reports:
ERROR: Method "editDefaultResponseFeedback" is defined in both
exploration-editor.ts and practice-question-submitter.ts.
Rename to follow the convention: editDefaultResponseFeedbackIn
ExplorationEditorPage / editDefaultResponseFeedbackInQuestionEditorPage

Unit tests in `other_files_linter_test.py` will cover:

- No duplicates across files → passes
- Same method name in two user files → fails with correct error identifying both files
- Methods in non-user utility files are excluded from the check

Part 2 – Component-level utility files:

Common actions shared across multiple user roles will be extracted into shared modules under `core/tests/playwright-acceptance-tests/utilities/common/`. For example, actions involving the state editor that are currently duplicated between `exploration-editor.ts` and `practice-question-submitter.ts` will be moved to `state-editor-utils.ts`:

```
// utilities/common/state-editor-utils.ts
export async function editDefaultResponseFeedback(
  page: Page, feedback: string
): Promise<void> {
  await page.click(defaultResponseTab);
  await page.fill(stateContentInputField, feedback);
  await page.click(saveOutcomeFeedbackButton);
}
```

Role files then call this shared function with a disambiguating wrapper that encodes the page context in the name, following the existing convention `{action}In{PageContext}Page`:

```
// utilities/user/exploration-editor.ts
import * as StateEditorUtils from '../common/state-editor-utils';

async editDefaultResponseFeedbackInExplorationEditorPage(
  feedback: string
): Promise<void> {
  await StateEditorUtils.editDefaultResponseFeedback(this.page, feedback);
}
```

This ensures there is only one implementation of each shared action, and the lint check in Part 1 enforces that no two role files ever define the same method name going forward.

This fix applies to the Playwright utilities directory since the Puppeteer directory is being removed as part of this project's migration. The lint check will point to

`playwright-acceptance-tests/utilities/user/` rather than
`puppeteer-acceptance-tests/utilities/user/`.

Third-Party Libraries

No.	Third-party library name and version	Link to third-party library	Why it is needed	License (if third-party library)	[Android only] Min / target / max SDK version that the library supports
1	Playwright	https://playwright.dev/	Used as the new framework for acceptance tests to improve stability and debugging	Apache 2.0	

...

"Service" Dependencies

No.	Dependency name	Why it is needed	What our plan is, if the dependency fails under us
1	GitHub Actions CI	Used to run acceptance tests and stress tests in CI	Local test execution and logs can be used for debugging if CI fails

Key High-Level and Architectural Decisions

Decision 1: How to support Puppeteer + Playwright together

I have considered the following alternatives:

1. **Big-bang migration - Migrate all tests at once**
2. **Dual-framework support - Allow both frameworks during migration**

Among these, I believe that 2nd is the best approach, because:

- Safer, does not break existing tests
- Allows incremental migration
- Easier to debug issues during transition
- Matches Oppia's guidance

The above approaches are contrasted in detail in the following table:

	Alternative 1: Big-bang migration	Alternative 2: Dual-framework support
Risk	High	Low

CI stability	Risky	Stable
Debugging	Hard	Easier
Incremental progress	No	Yes

Risks and mitigations

Potential Risk	Mitigation
Isolated Node 20 runtime path not found at startup	Path validated in <code>install_playwright_dependencies()</code> at startup with a clear error message identifying the missing binary path and the expected location at <code>oppia_tools/node-20.11.1/</code>
Isolated Node 20 installation conflicting with existing oppia_tools structure	Node 20 is installed in its own versioned subdirectory (<code>node-20.11.1/</code>) with no overlap with existing tools; existing Node 16 path constants in <code>common.py</code> are unchanged
Breaking CI during migration	Dual-framework support maintained throughout; each migrated suite is validated in CI before proceeding to the next
Conflicts in shared utilities	Codeowners would ensure that the PRs changing the shared methods(in puppeteer and playwright) makes sure that both copies are in sync.
Flakes not reproducible	Use stress test workflow (<code>stress_test_acceptance_test.yml</code>) to reproduce consistently across multiple CI runs
Migration introducing regressions	Perform minimal-change migration first (preserve existing test logic and selectors); validate each suite using CI before merging
CI becoming slower	Limit stress testing scope; optimize artifact uploads; Playwright's built-in parallelism reduces per-suite overhead

Key Objectives for Each Milestone

Milestone	Key Objective
Milestone 1	Infrastructure PRs (1–7) establish: isolated Node 20 runtime, Playwright project structure, framework-aware routing via <code>acceptance.json</code> , dual-directory CI scanning, <code>servers.py</code> routing, CI artifact uploads, and lint enforcement for

	duplicate method names. Additionally 2 e2e tests migrated to Puppeteer acceptance tests (PR 9). Acceptance tests can be executed using the Playwright framework through the existing test runner (<code>python -m scripts.run_acceptance_tests</code>), with <code>logged-in-learner</code> test suites (PRs 7.1 to 7.9 and 7.16) and exploration-creator test suites (PRs 7.10 to 7.15) migrated from Puppeteer to Playwright by Jun 26. CI workflows run acceptance tests using Playwright without relying on Puppeteer for these suites.
Milestone 2	All <code>logged-out-learner</code> test suites (PRs 1–13) migrated from Puppeteer to Playwright by Aug 2. Fix method overriding issue in user-factory.ts. Acceptance tests provide improved debugging support including: (1) complete execution even on snapshot mismatches with updated snapshots uploaded as CI artifacts (PR 14, Aug 5–8); (2) a published debugging guide covering local debugging, Playwright Inspector, Trace Viewer, CI artifacts, and snapshot updates (PR 16, Aug 16–19).

Implementation Approach

[Web only] Storage Model Layer Changes

N/A

[Web only] Storage Model Migrations

N/A

Domain Objects

N/A

User Flows (Controllers and Services)

N/A

[Web only] Web frontend changes

N/A

Documentation changes

A debugging guide will be written and published to the Oppia developer documentation (oppia-web-developer-docs repository, tracking issue [#422](#)). The guide will cover:

- How to run acceptance tests locally using `python -m scripts.run_acceptance_tests`

- How to use Playwright Inspector (`npm run playwright test --debug`) to step through test actions interactively
- How to use Playwright Trace Viewer (`npm run playwright show-trace`) to inspect DOM snapshots, network calls, and console logs at the point of failure
- How to download and interpret CI artifacts (videos, traces, screenshots) from failed runs
- How to trigger and interpret the stress test workflow (`stress_test_acceptance_test.yml`) to reproduce and validate fixes for flaky tests
- How to update snapshots when UI changes are intentional

This addresses tracking issues [#16136](#), [#22319](#), and [#422](#).

Metrics Plan

Event (see PRD)	Event parameters (see PRD)	Do we already record the event + parameters? • If so, please link to the corresponding code on GitHub. • If not, describe the changes needed to do so.
CI debugging artifacts present on every acceptance test run	Traces present on failure (bool); screenshots present on failure (bool); video present for all mobile suite runs (bool); updated snapshot uploaded as artifact on mismatch (bool)	Partially. The existing <code>run-a-specific-acceptance-test/action.yml</code> already uploads some Puppeteer artifacts. Changes needed: (1) add Playwright artifact upload steps for <code>test-results/</code> (Section 13.3); (2) fix mobile recording via <code>video: 'on'</code> globally in <code>playwright.config.ts</code> (Section 13.7, issue #23155); (3) upload new snapshots as artifacts on mismatch (Section 13.7, issue #23776). Validated via CI artifacts generated during stress test runs.
Snapshot mismatch no longer fast-fails mid-test	Test continues executing all steps after a snapshot mismatch (bool); all mismatch errors reported together in final output (bool); updated snapshot uploaded as CI artifact on mismatch (bool)	Not yet. Implemented in Milestone 2 via the <code>snapshotErrors</code> array in <code>fixtures.ts</code> (Section 13.7, issue #23776). Verified by introducing a controlled snapshot mismatch in a test and confirming all subsequent steps execute before the error is thrown, and the new snapshot appears in CI artifacts.
Duplicate method name conflict in <code>UserFactory</code> detected by CI lint check	Duplicate method name (string); files where it appears (list of filenames); CI lint check fails on detection (bool)	Not yet. Implemented via a new lint rule in <code>other_files_linter.py</code> (PR 7, M1) that scans <code>.ts</code> files under <code>playwright-acceptance-tests/utilities/user/</code> and fails CI if the same async method name appears in more than one file. Target: 0 silent method

		overrides in the Playwright utilities directory by end of M1.
--	--	---

Implementation Plan

Milestone Table (include both PRs and other actions that need to be taken prior to launch)

Milestone 1

Key objective for this milestone: *Initialize Playwright infrastructure alongside Puppeteer, enable dual-framework execution, and migrate logged-in-learner user test specs from Puppeteer to Playwright while maintaining CI stability. Also migrate 2 e2e tests to puppeteer acceptance tests.*

No.	Description of PR / action	Sub-tasks (1–2 day breakdown)	Prerequisite steps	Target date	
				PR creation	PR merge
1	Install isolated Node 20 runtime under <code>oppia_tools</code> ; add <code>PLAYWRIGHT_NODE_PATH</code> , <code>PLAYWRIGHT_NPM_BIN_PATH</code> , <code>PLAYWRIGHT_NPX_BIN_PATH</code>	Day 1 (May 24): Add constants to <code>common.py</code> ; implement <code>install_playwright_node()</code> in <code>install_third_party_libs.py</code>	None	May 25	May 26

	H, <code>PLAYWRIGHT_NODE_VERSION</code> constants to <code>scripts/common.py</code>; add <code>install_playwright_node()</code> to <code>install_third_party_libs.py</code>; Add unit tests to <code>common_test.py</code> and <code>install_third_party_libs_test.py</code>	Day 2 (May 25): Write unit tests; verify existing CI workflows remain stable			
2	Extend <code>acceptance.json</code> schema to include <code>framework</code> field; Add <code>install_playwright_dependencies()</code> to <code>run_acceptance_tests.py</code>; add framework-aware compilation guard for <code>compile_test_ts_files()</code> (skip for playwright suites); inline framework reading logic in <code>run_tests()</code>; add unit tests.	Day 1 (May 26): Extend <code>acceptance.json</code> , Implement <code>install_playwright_dependencies()</code> and compilation guard Write unit tests in <code>run_acceptance_tests_test.py</code>	Node 20 runtime installed	May 27	Jun 2
3	Implement framework-aware routing in <code>servers.py</code> using the local <code>node_modules/.bin/playwright</code> binary; integrate with <code>run_acceptance_tests.py</code>; add unit tests in <code>servers_test.py</code>	Day 1 (May 26): Implement routing logic in <code>servers.py</code> ; integrate with <code>run_acceptance_tests.py</code> Write unit tests in <code>servers_test.py</code>	Configuration schema updated	May 27	Jun 2
4	Update <code>check_tests_are_captured_in_ci.py</code> to scan both spec directories; add unit tests	Day 1 (May 27): Extend schema; update scanning logic in <code>check_tests_are_captured_in_ci.py</code> ; write unit tests in <code>check_tests_are_captured_in_ci_test.py</code>	Playwright setup complete	May 27	Jun 2
5	Update <code>run-a-specific-acceptance-test/action.yml</code> to add Playwright artifact upload steps to the existing	Day 1 (May 28): Update <code>action.yml</code> ; add Playwright artifact upload steps; test on a branch	Routing logic implemented	May 29	Jun 3

	Puppeteer video upload step; support dual-framework artifact uploads				
6	Migrate 2 e2e tests to acceptance tests using Puppeteer. The remaining 5 e2e tests have open PR's that are either marked LGTM or being actively worked on by members of dev-workflow team. Will help them solve CI errors and get their PR's merged.	Day 1–2 per webdriverio suite: migrate spec to Puppeteer CUJ structure; verify locally and in CI	No prerequisites	May 31	Jun 6
6.1	Migrate e2e test navigation.js - issue https://github.com/oppia/oppia/issues/24606	Day 1 (May 30): Migrate the test. Verify stability using stress tests.	No prerequisites	May 30	Jun 4
6.2	Migrate e2e test publicationAndLibrary.js - issue https://github.com/oppia/oppia/issues/25060	Day 1 (May 31): Migrate the test. Day 2 (Jun 1): Verify stability using stress tests.	No prerequisites	Jun 1	Jun 5
–	Checkpoint: Dual-framework execution working (Puppeteer + Playwright) with stable CI				
	<u>Demo to PMs/stakeholders:</u> - Acceptance tests can run using both Puppeteer and Playwright - Framework-based routing via acceptance.json works - CI successfully executes both frameworks - All e2e tests migrated to acceptance tests		CI and routing setup	Jun 6	
7	Start migrating puppeteer acceptance tests to playwright acceptance tests. Create playwright project folder under <code>core/tests/playwright-acceptance-tests/</code> with <code>package.json</code> ,		Playwright migration workflow established		

	<code>playwright.config.ts</code> , <code>.gitignore</code> ; install dependencies;				
7.1	Migrate <code>logged-in-learner/add-and-remove-exploration-from-play-later</code>	Day 1 (Jun 4): Migrate the test; verify locally. Run in CI; fix failures.		Jun 4	Jun 8
7.2	Migrate <code>logged-in-learner/completes-the-exploration-and-decides-what-to-do-next</code>	Day 1 (Jun 5): Migrate the test; verify locally. Run in CI; fix any failures.		Jun 5	Jun 9
7.3	Migrate <code>logged-in-learner/sets-goal-on-learner-dashboard</code>	Day 1 (Jun 6): Migrate the test; verify locally. Run in CI; fix any failures.		Jun 6	Jun 10
7.4	Migrate <code>logged-in-learner/provide-feedback-on-the-lesson-or-report-it-from-the-lesson-player</code>	Day 1 (Jun 7): Migrate the test; verify locally. Run in CI; fix any failures.		Jun 7	Jun 11
7.5	Migrate <code>logged-in-learner/access-pages-that-require-higher-privileges</code> and <code>logged-in-learner/goes-through-the-sign-in-flow</code>	Day 1 (Jun 8): Migrate both the tests; verify locally. Run in CI; fix any failures.		Jun 8	Jun 12
7.6	Migrate <code>logged-in-learner/reaches-a-checkpoint-and-saves-their-progress</code>	Day 1 (Jun 9): Migrate both the tests; verify locally. Run in CI; fix any failures.		Jun 9	Jun 13
7.7	Migrate <code>logged-in-learner/export-and-delete-account</code> and <code>logged-in-learner/changes-site-language-to-rtl</code>	Day 1 (Jun 10): Migrate both the tests; verify locally. Run in CI; fix any failures.		Jun 10	Jun 14
7.8	Migrate and <code>logged-in-learner/edit-the-profile</code>	Day 1 (Jun 11): Migrate the test; verify locally. Run in CI; fix any failures.		Jun 11	Jun 15

7.9	Migrate logged-in-learner/choose-new-lesson-to-play-from-learner-dashboard	Day 1 (Jun 12): Migrate the test; verify locally. Run in CI; fix any failures.		Jun 12	Jun 16
7.10	Migrate logged-in-learner/manage-classroom-progress-in-home-learner-dashboard	Day 1 (Jun 13): Migrate the test; verify locally. Day 2 (Jun 14): Run in CI; fix any failures.		Jun 14	Jun 18
7.11	Migrate logged-in-learner/manage-in-progress-and-completed-lessons-and-skill	Day 1 (Jun 15): Migrate the test; verify locally. Day 2 (Jun 16): Run in CI; fix any failures.		Jun 16	Jun 20
7.12	Migrate logged-in-learner/manage-in-progress-and-completed-community-lessons-and-skill	Day 1 (Jun 17): Migrate the test; verify locally. Day 2 (Jun 18): Run in CI; fix any failures.		Jun 18	Jun 22
7.13	Migrate logged-in-learner/manage-goals-in-learner-dashboard	Day 1 (Jun 19): Migrate the test; verify locally. Day 2 (Jun 20): Run in CI; fix any failures.		Jun 20	Jun 24
7.14	Migrate logged-in-learner/interact-with-goals-in-learner-dashboard	Day 1 (Jun 21): Migrate the test; verify locally. Day 2 (Jun 22): Run in CI; fix any failures.		Jun 22	Jun 26
7.15	Migrate logged-in-learner/manage-community-lesson-progress-in-home-learner-dashboard	Day 1 (Jun 23): Migrate the test; verify locally. Run in CI; fix any failures.		Jun 23	Jun 27
7.16	Migrate logged-in-learner/starts-from-beginning-after-completing-a-lesson	Day 1 (Jun 24): Migrate the test; verify locally.		Jun 24	Jun 28
	<u>Demo to PMs/stakeholders:</u> The above-mentioned tests of logged-in-learner users run through playwright.		Complete migration of above tests.	Jun 26	

Note: Weeks 6–8 (Jun 27–Jul 17) cover Oppia's internal M1 testing period, buffer time for revisions, and the official GSoC midpoint evaluation. These are not active coding weeks

Milestone 2

Key objective for this milestone: *Migrate logged-out-learner user tests specs, fix method overriding issue and snapshot non-fast fail behaviour and document debugging guides.*

No	Description of PR / action	Sub-tasks (1–2 day breakdown)	Prerequisite steps	Target date	
				PR creation	PR merge
1	Migrate <code>logged-out-learner/play-a-complete-community-lesson</code>	Day 1 (Jul 17): Migrate the test; verify locally. Day 2 (Jul 18): Run in CI; fix failures.		Jul 18	Jul 20
2	Migrate <code>logged-out-learner/listen-to-voiceovers-of-the-lessons-in-the-lesson-player</code>	Day 1 (Jul 19): Migrate the test; verify locally. Day 2 (Jul 20): Run in CI; fix failures.		Jul 20	Jul 22
3	Migrate <code>logged-out-learner/complete-some-practice-question</code>	Day 1 (Jul 21): Migrate the test; verify locally. Run in CI; fix any failures.		Jul 21	Jul 23
4	Migrate <code>logged-out-learner/reaches-a-checkpoint-and-saves-their-progress</code>	Day 1 (Jul 22): Migrate the test; verify locally. Run in CI; fix any failures.		Jul 22	Jul 24
5	Migrate <code>logged-out-learner/use-the-feedback-and-help-cards</code>	Day 1 (Jul 23): Migrate the test; verify locally. Run in CI; fix any failures.		Jul 23	Jul 25
6	Migrate <code>logged-out-learner/share-the-lesson-from-the-lesson-player</code>	Day 1 (Jul 24): Migrate the test; verify locally. Run in CI; fix any failures.		Jul 24	Jul 26

7	Migrate <code>logged-out-learner/give-feedback-on-the-lesson-from-the-lesson-player</code>	Day 1 (Jul 25): Migrate the test; verify locally. Run in CI; fix any failures.		Jul 25	Jul 27
8	Migrate <code>logged-out-learner/disc-over-the-website-and-navigate-to-math-classroom</code>	Day 1 (Jul 26): Migrate the test; verify locally. Run in CI; fix any failures.		Jul 26	Jul 28
9	Migrate <code>logged-out-learner/pick-a-lesson-to-learn</code>	Day 1 (Jul 27): Migrate the test; verify locally. Run in CI; fix any failures.		Jul 27	Jul 29
10	Migrate <code>logged-out-learner/search-for-a-specific-exploration</code>	Day 1 (Jul 28): Migrate the test; verify locally. Run in CI; fix any failures.		Jul 28	Jul 30
11	Migrate <code>logged-out-learner/visits-the-community-library-and-look-around</code>	Day 1 (Jul 29): Migrate the test; verify locally. Run in CI; fix any failures.		Jul 29	Jul 31
12	Migrate <code>logged-out-learner/access-the-lesson-player</code>	Day 1 (Jul 30): Migrate the test; verify locally. Run in CI; fix any failures.		Jul 30	Aug 1
13	Migrate <code>logged-out-learner/complete-the-embedded-lesson</code>	Day 1 (Jul 31): Migrate the test; verify locally. Run in CI; fix any failures.		Jul 31	Aug 2
14	Fix snapshot non-fast-fail: update <code>fixtures.ts</code> with <code>snapshotErrors</code> array, error collection in <code>expectScreenshotToMatch</code> , fixture teardown throwing combined error after all steps complete; upload new snapshots as CI artifacts	Day 1 (Aug 3): Implement <code>snapshotErrors</code> array and error collection in <code>fixtures.ts</code> Day 2 (Aug 4): Implement fixture teardown logic; write controlled test to verify behavior; add CI artifact upload	Playwright-based test execution working	Aug 4	Aug 8
15	Fix method overriding issue in UserFactory (detailed in Section 13.9)	Day 1 (Aug 8,9): Implement duplicate method detection; fix any existing violations Day 2 (Aug 10): Write unit tests	Stable Playwright-based test execution	Aug 10	Aug 14

16	Create debugging guide in oppia-web-developer-docs covering: running tests locally, Playwright Inspector, Trace Viewer, CI artifacts, stress test workflow, updating snapshots (tracking issues #16136, #22319,)	Day 1 (Aug 15): Write sections on local debugging, Playwright Inspector, Trace Viewer Day 2 (Aug 16): Write sections on CI artifacts, stress test workflow, snapshot updates; submit PR for review	CI artifacts and tracing enabled	Aug 16	Aug 18
	<u>Demo to PMs/stakeholders:</u> - Tests complete execution even with snapshot mismatches - All logged-out-learner mentioned above migrated to playwright. - Debugging guide published		Stable Playwright execution, debugging setup	Aug 19	

Feature Flags

#	Name of feature flag	What happens when the flag is turned on	Needs sync between Android / Web?	Key stakeholders
1.				
2.				
3.				

Future Work

Note: This section is mainly for reference (since items listed here won't be part of the GSoC project).
Proposals will primarily be evaluated based on the implementation plan above.

- Further optimize CI performance and parallelization
- Improve test coverage and add new acceptance tests
- Enhance debugging tooling and visualization

...

Appendix

Section 13.1

The following Playwright-specific constants will be added to `scripts/common.py` (additive only, no existing constants changed):

```
PLAYWRIGHT_NODE_VERSION = '20.11.1'
PLAYWRIGHT_NODE_PATH = os.path.join(
    OPPIA_TOOLS_DIR, 'node-%s' % PLAYWRIGHT_NODE_VERSION)
PLAYWRIGHT_NPM_BIN_PATH = os.path.join(
    PLAYWRIGHT_NODE_PATH, 'bin', 'npm')
PLAYWRIGHT_NPX_BIN_PATH = os.path.join(
    PLAYWRIGHT_NODE_PATH, 'bin', 'npx')
```

A Playwright configuration file will be introduced:

```
import {defineConfig, devices} from '@playwright/test';

const isMobile = process.env.MOBILE === 'true';

export default defineConfig({
  testDir: './specs',
  timeout: 120000,
  expect: { timeout: 10000 },
  fullyParallel: false,
  reporter: 'list',
  use: {
    baseURL: 'http://localhost:8181',
    headless: process.env.HEADLESS !== 'false',
    trace: 'retain-on-failure',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
  },
  projects: isMobile
    ? [{ name: 'mobile-chrome', use: {...devices['Pixel 7']} }]
    : [{ name: 'chromium', use: {...devices['Desktop Chrome']} }],
});
```

Section 13.2.1

```

{
  "suites": [
    {
      "name": "logged-in-learner/serial-chapter",
      "module": "core/tests/playwright-acceptance-tests/specs/
logged-in-learner/serial-chapter.spec.ts",
      "framework": "playwright"
    },
    {
      "name": "interested-donor/make-a-donation",
      "module": "core/tests/puppeteer-acceptance-tests/specs/
interested-donor/make-a-donation.spec.ts",
      "framework": "puppeteer"
    }
  ]
}

```

Section 13.2.3

```

suite_config = available_suites[suite_name]
module = suite_config['module']
framework = suite_config['framework']

if framework == 'puppeteer':
    acceptance_tests_args = [
        nodemodules_jest_bin_path,
        module,
        '--config=./core/tests/puppeteer-acceptance-tests/jest.config.js',
    ]
elif framework == 'playwright':
    playwright_bin = os.path.join(
        common.CURR_DIR,
        'core', 'tests', 'playwright-acceptance-tests',
        'node_modules', '.bin', 'playwright'
    )
    acceptance_tests_args = [
        playwright_bin,
        'test',
        module,

```

```

    '--config=./core/tests/playwright-acceptance-tests/playwright.config.ts',
  ]
else:
  raise Exception(
    f'Invalid framework for suite {suite_name}: {framework}')

```

Section 13.6

Flake root cause categories and fixes:

Category	Symptoms	Fix strategy
Frontend race conditions / logic bugs	Test fails intermittently even with correct waits/selectors UI behaves inconsistently (e.g., button not enabled, data missing) Reproducible manually sometimes	Investigate frontend async flows (Promises, state updates, API calls) Fix race conditions in code instead of adding waits Add proper state synchronization in frontend (e.g., ensure data loaded before rendering)
Timing issues	Passes locally, fails in CI intermittently	Replace manual waits with Playwright auto-waiting; use <code>expect(locator).toBeVisible()</code> instead of fixed delays
Unstable selectors	Element not found errors on specific runs	Switch to role-based or <code>data-testid</code> selectors stable across renders
Inconsistent UI state	Test fails at different steps across runs	Add explicit state assertions before interactions
Backend/network latency	API responses arriving late	Use <code>waitForResponse</code> to wait for specific network calls before proceeding